Testbed Setup & Configuration Plan



ACE

CYBER R&D Program

Conversational AI Interface for
Analyst-Driven Threat Investigation

Koi McFarland
Charleston Southern University
2025

Testbed Setup & Configuration Plan

Document ID:	TSCP-001
Document Title:	Testbed Setup & Configuration Plan
Project Title:	Conversational AI Interface for Analyst-Driven Threat Investigation
Version #:	1.4
Status:	Draft
Author:	Koi McFarland
Date:	2025-10-29

1.0 Testbed Purpose and Scope

The research testbed provides a structured and controlled environment for evaluating the effectiveness of a Conversational AI interface designed to assist SOC analysts in querying Elasticsearch log data using natural language. It is designed to be reproducible, secure, and aligned with the project's research objectives, focusing on log querying, anomaly identification, and analyst decision support. The Conversational AI component uses a locally hosted Large Language Model (Mistral running on Ollama) to translate natural-language queries into Elasticsearch JSON DSL. A rule-guided prompt template, schema constraints, and a lightweight validation layer ensure that generated queries are syntactically valid and safe before execution. All AI-generated queries are recorded in an audit log, and guardrails are implemented to mitigate prompt-injection risks, unsafe query patterns, ambiguous queries, and attempts at large-scale data exfiltration.

Correct threat identification in this study is defined as successfully retrieving events that match pre-tagged indicators in the synthetic dataset. These tags are based on explicit, objective classification criteria (e.g., "failed login attempts above a defined threshold from high-risk IP ranges"), ensuring consistent scoring.

Due to project time constraints and the need to focus on measurable model-driven performance metrics, this study does not compare the Conversational AI interface against human SOC analysts; instead, it evaluates the CAI solely against pre-tagged synthetic threat events using objective accuracy and F1-score criteria.

1.1 Functional Scope

The testbed will simulate the key SIEM capabilities relevant to Elasticsearch Enterprise, including:

- Alert correlation and enrichment: Events from multiple sources are aggregated and enriched with metadata for better detection accuracy.

- Rule-based detection and tuning: Configurable detection rules simulate threat-hunting scenarios for failed logins, suspicious IPs, and lateral movement.
- Search and retrieval: Analysts can query logs using both standard query languages (KQL/Lucene) and the Conversational AI interface.
- Visualization and triage support: Kibana dashboards provide an interactive interface to inspect alerts, supporting simulated SOC workflows.

1.2 Research Scope

The testbed enables systematic evaluation of research variables under controlled conditions:

- Independent Variables:
  - Threat-hunting task type (e.g., failed logins, suspicious IPs)
- Dependent Variables:
  - Query accuracy (F1 comparison to ground-truth JSON DSL)
  - Detection accuracy (percentage of correctly retrieved pre-tagged threats)
- Control Variables:
  - Use of pre-tagged synthetic logs with objective labeling criteria
  - Fixed workload, data volume, and network environment

Ground-truth reference queries are derived directly from the same objective labeling rules used in the synthetic log generator. Each threat type (e.g., brute force, failed logins, lateral movement) has a corresponding canonical Elasticsearch JSON DSL query that exactly implements the tagging criteria. These queries are validated by cross-checking their returned event UUIDs against the pre-tagged /pi-key mapping and stored as version-controlled JSON artifacts to ensure reproducibility. This approach guarantees that F1 calculations are based on precise, deterministic, and auditable definitions of what constitutes a correct query.

1.3 Success Criteria & Benchmarks

- Query Accuracy: AI-generated queries should achieve an F1 score ≥ 0.90 when compared to ground-truth reference queries.
- Detection Accuracy: At least 95% of pre-tagged suspicious log events should be correctly identified by AI-generated queries.
- Safety: No AI-generated query should violate safety constraints (prompt-injection attempts, unsafe query expansions, or excessive data retrieval).

1.4 Operational Workflow Simulation

The testbed emulates a simplified SOC workflow:

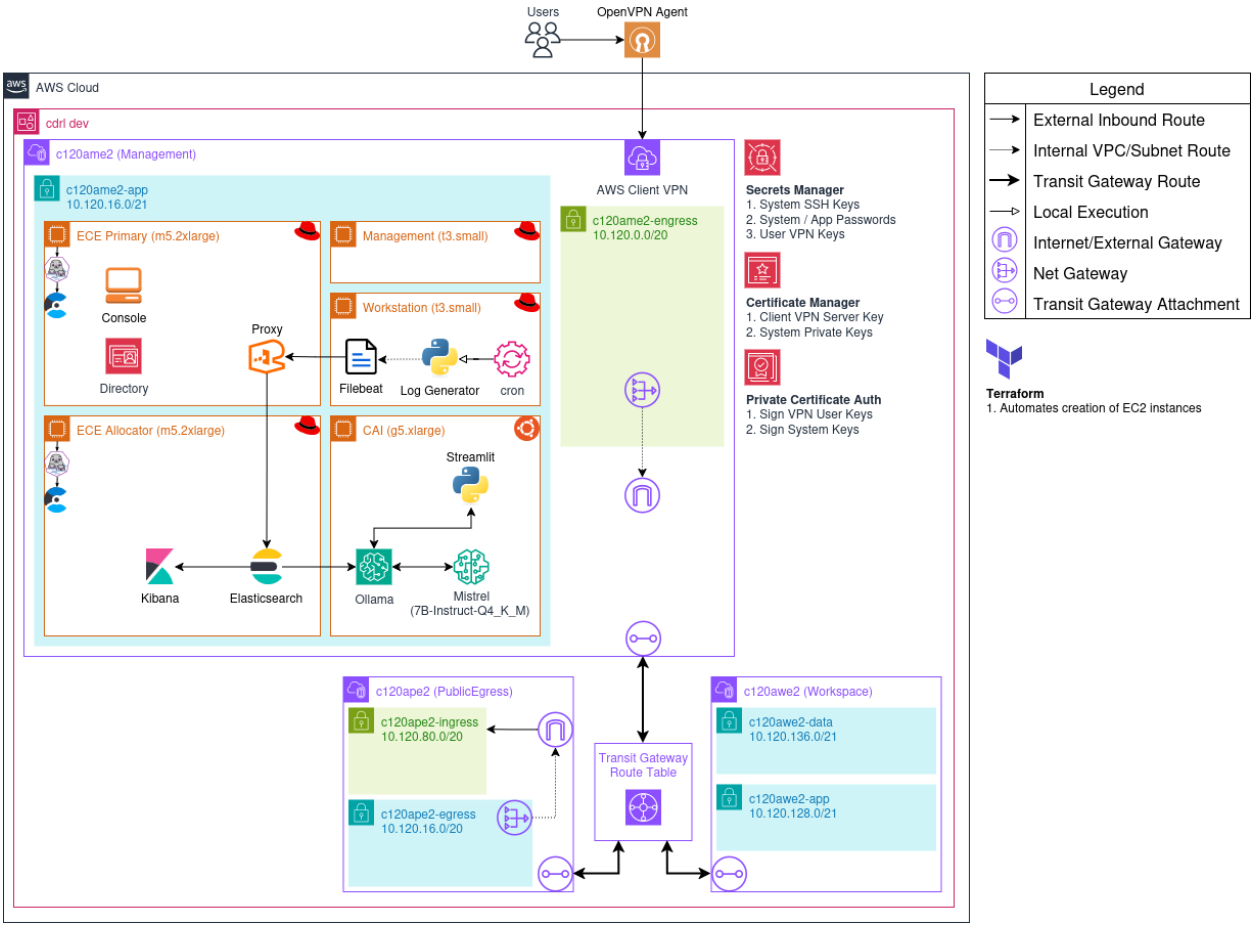
- Log ingestion: Filebeat forwards synthetic log events into Elasticsearch for indexing.
- Detection and correlation: Configurable detection rules simulate alert generation and anomaly detection.
- Analyst interaction: Both manual queries and the Conversational AI interface return results through the analyst workstation UI, where results are summarized, timestamped, and logged.
- Evaluation: All CAI-generated queries and Elasticsearch responses are recorded for auditing, quantitative scoring, error analysis, and validation against ground-truth labels.

By combining structured functional behavior with controlled research measurement, the testbed creates a realistic yet manageable platform for evaluating the performance and safety of a Conversational AI interface in SOC-oriented log analysis tasks.

2.0 Testbed Architecture Overview

The testbed consists of multiple interconnected EC2 instances and Podman containers. The primary components include:

- **Elastic Cloud Enterprise:** Comprising Elasticsearch (data indexing and storage), Kibana (visualization and dashboarding), and Filebeat (log forwarding).
- **Log Simulation Node:** Generates synthetic log data and simulated attack events (e.g., brute force, privilege escalation).
- **Analyst Workstation:** Provides a controlled interface for monitoring alerts and visualizing data.



Component	Function	Host Type	OS / Version	Network Role
AWS Cloud	Hosts the entire Testbed Cloud Platform	Cloud Platform	N/A	Global
EC2 Service	Manages Virtual Machines (Instances)	Cloud Compute Service	N/A	Global
Terraform	Automates provisioning and configuration of AWS	Infrastructure Tool	N/A	Global

Component	Function	Host Type	OS / Version	Network Role
	resources (EC2, VPC, subnets, IAM, etc.).			
VPC	Defines isolated virtual networks and subnets for management and data plane separation.	Cloud Networking Service	N/A	All Subnets
ECE Allocator Node	Manages container deployment and resource allocation via the Elastic Container Orchestrator.	EC2 Instance	RHEL 8.10 AMI	Management Subnet (10.120.16.0/21)
ECE Primary Node	Hosts Elastic Cloud Enterprise core services: Orchestrator, Management, Proxy, and Director.	EC2 Instance	RHEL 8.10 AMI	Management Subnet (10.120.16.0/21)
Podman	Container runtime managing Elastic Stack components (Elasticsearch,	Container Platform	RHEL 8.10 (on ELK EC2)	Management Subnet (10.120.16.0/21)

Component	Function	Host Type	OS / Version	Network Role
	Kibana).			
Elasticsearch	Indexes, stores, and retrieves SOC log data for querying and analysis.	Podman Container	Elastic Image (on ELK EC2)	Management Subnet (10.120.16.0/21)
Kibana	Provides the web-based dashboard and visualization interface for Elasticsearch data.	Podman Container	Elastic Image (on ELK EC2)	Management Subnet (10.120.16.0/21)
Workstation Node	Generates synthetic SOC logs and runs Filebeat and automation scripts.	EC2 Instance	RHEL 8.10	Management Subnet (10.120.16.0/21)
Filebeat	Ships generated logs to Elasticsearch through the ingest pipeline.	Lightweight Agent	RHEL 8.10 (on Log Generator EC2)	Management Subnet (10.120.16.0/21)
Crontab	Schedules recurring execution of the log	System Scheduler	RHEL 8.10 AMI	Management Subnet (10.120.16.0/21)

Component	Function	Host Type	OS / Version	Network Role
	generator script (e.g., every 5 minutes).			
Log Generator	Python script that generates labeled SOC logs (benign/adversarial) for ingestion.	Python Script	RHEL 8.10 (on Log Generator EC2)	Management Subnet (10.120.16.0/21)
Conversational AI Interface	Hosts the LLM-based interface that allows analysts to query logs via natural language.	EC2 Instance	Ubuntu 22.04	Management Subnet (10.120.16.0/21)
Ollama	Local LLM hosting and model serving engine for the CAI.	AI Runtime	Ubuntu 22.04	Management Subnet (10.120.16.0/21)
Mistral 7B-Instruct-Q4_K_M	Lightweight large language model powering the CAI's natural language understanding	LLM Model	Ubuntu 22.04	Management Subnet (10.120.16.0/21)
Streamlit	Front-end web	Python	Ubuntu 22.04	Management

Component	Function	Host Type	OS / Version	Network Role
	application for the Conversational AI Interface.	Framework		Subnet (10.120.16.0/21)

3.0 Hardware and Virtual Resources

Resource	Description	Specs	Allocation
AWS Cloud	Hosts the entire Testbed Cloud Platform	N/A	Global Platform
ECE Primary Node	Hosts Elastic Cloud Enterprise core services: Orchestrator, Management, Proxy, and Director.	AMI: RHEL 8.10 Storage: 150 GiB EBS vCPUs: 4 Memory: 16 GiB	Instance Type: m5.xlarge
ECE Allocator Node	Manages container deployment and resource allocation via the Elastic Container Orchestrator.	AMI: RHEL 8.10 Storage: 150 GiB EBS vCPUs: 4 Memory: 16 GiB	Instance Type: m5.xlarge
Workstation Node	Generates synthetic SOC logs and runs Filebeat and automation scripts.	AMI: RHEL 8.10 Storage: 20 GiB EBS vCPUs: 2 Memory: 2 GiB	Instance Type: t3.small
Conversational AI Interface Node	Hosts the LLM-based interface that allows	AAMI: Ubuntu 22.04 Storage: 40 GiB EBS vCPUs: 4	Instance Type: g5.xlarge

Resource	Description	Specs	Allocation
	analysts to query logs via natural language.	Memory: 16 GiB GPU: NVIDIA (G5 instance family)	
GitHub	Project Manager with versioning control	N/A	External Service

4.0 Software Stack and Tools

Provide details on all software tools installed.

Component	Purpose	Installation Method	Version
Elasticsearch	Stores and indexes SOC log data for analysis.	Podman Pull	9.1.5
Kibana	Provides visualization and query interface for Elasticsearch data.	Podman Pull	9.1.5
Logstash	Ingests and processes log data before indexing in Elasticsearch.	Podman Pull	9.1.5
Filebeat	Ships generated SOC logs to Logstash or Elasticsearch.	Podman Pull	9.1.5
Terraform	Automates provisioning of AWS infrastructure resources.	dnf	1.13.3
Podman	Manages containerized services and Elastic Stack components.	dnf	28.5.1
Python 3	Executes log-generation scripts and CAI frontend logic.	dnf	3.11
Faker	Generates synthetic (benign/adversarial) log data.	dnf	37.11.0
Streamlit	Web-based frontend for the Conversational AI Interface.	apt	1.39.0
Ollama	Hosts and serves local LLMs	curl	0.4.2

Component	Purpose	Installation Method	Version
	for natural language processing.		
Mistral	Lightweight instruction-tuned LLM used by the CAI for SOC query interpretation.	Ollama Pull	7B-Q4_K_M

5.0 Network Configuration

5.1 Different Virtual Networks

Network	Subnet	Type	Purpose
c120ame2	10.120.0.0/18	VPC	Management
c120ame2-support	10.120.16.0/20	Subnet	Workload / Testbed
c120ame2-egress	10.120.0.0/20	Subnet	Outbound Internet Access
c120awe2	10.120.128.0/18	VPC	Unused
c120ape2	10.120.64.0/18	VPC	Unused

5.2 Protocol / IP / Port List

Source	Destination	Protocol	Port	Purpose
Management Subnet (10.120.16.0/20)	All EC2 instances	TCP	22	SSH admin access to internal EC2 hosts
Workstation EC2 (Log Generator)	Filebeat	Localhost	N/A	Python log generator writes logs; Filebeat reads directly from file. (No network port needed.)
Workstation EC2 (Filebeat)	Elasticsearch (on ECE Primary via Podman)	TCP	9200	Filebeat sends logs directly into Elasticsearch ingest pipeline.
Kibana (Podman	Elasticsearch	TCP	9200	Kibana queries Elasticsearch

Source	Destination	Protocol	Port	Purpose
on ECE Primary)	(Podman on same host)			for dashboards and data retrieval.
Analyst Workstation (External)	Kibana (ECE Primary Public/Proxy Endpoint)	HTTPS	443	Analyst accesses Kibana UI for log visualization.
Analyst Workstation (External)	Conversational AI Interface (CAI EC2)	HTTPS	8501	Analyst queries CAI (Streamlit).
CAI Node (Streamlit App)	Ollama Runtime (local on same EC2)	HTTP (internal)	11434	Streamlit sends LLM prompts to Ollama for processing.
CAI Node (Ollama)	Mistral Model (local)	Local runtime	N/A	No networking; Mistral is served by Ollama directly.
CAI Node	Elasticsearch (ECE Primary)	TCP	9200	CAI translates natural-language queries and sends structured queries to Elasticsearch.
EC2 Instances (All)	Internet	TCP/UDP	80, 443	Outbound OS updates, Podman image pulls, Python/pip downloads, etc.

6.0 Security Controls

The AWS-based testbed will implement security measures to ensure isolation and integrity during experiments. Account and access control are managed by Core4ce through AWS IAM roles and policies, restricting access to specific users with multi-factor authentication (MFA) enabled, while EC2 instances run with least-privilege principles, using non-root users and IAM instance profiles for service interactions.

Encryption is enforced via AWS-managed keys for EBS volumes (using AES-256), TLS 1.3 for Kibana web interfaces and internal Elastic Stack communications, SSH with key-based authentication for instance access, and OpenVPN3 for secure remote management.

Network isolation is achieved through a dedicated VPC with a private subnet for management traffic, security groups acting as firewalls to allow only necessary ports, and no direct internet access for internal resources, simulating a sandboxed environment with NAT gateways for outbound traffic in the external subnet.

Logging and auditing are handled by AWS CloudTrail for API activity tracking, VPC Flow Logs for network monitoring, and integrated Elastic Stack logging for application-level events, with all logs centralized in encrypted S3 buckets for retention and analysis.

7.0 Test Data Integration

The AWS-based testbed data sources are integrated through a pipeline simulating SOC log workflows. Synthetic datasets, representing normal and adversarial activities, are generated using Python scripts on the Log Generator EC2 instance, using libraries such as Faker to create logs such as authentication events, failed logins, and suspicious IP accesses.

Logs are written to local paths like /var/log/generated/generated-soc-logs.ndjson. These logs are ingested using Filebeat configured to monitor the local path, parse events, and forward them over TLS-secured connections to the ECE Allocator EC2 instance for processing. They are then indexed into Elasticsearch, stored in date-partitioned indices such as soc-logs-YYYY.MM.DD for efficient querying.

Datasets are loaded automatically via cron-scheduled Python scripts that generate logs at configurable intervals, mimicking real-time traffic without using real datasets.

Validation steps include monitoring Filebeat metrics for successful shipments, querying Elasticsearch via curl/Kibana Dev Tools to confirm index population, and visualizing data in Kibana dashboards to verify flow, alert generation, and absence of ingestion errors. For example, synthetic alert data from Python scripts is streamed into Elastic via Filebeat for visualization in Kibana dashboards, allowing analysts to confirm threat patterns like brute-force attempts.

8.0 Configuration Management

The primary configuration tools for this project include Terraform, which is used to provision and manage AWS resources such as VPCs, EC2 instances, security groups, and subnets, and Git, which provides version control for all custom scripts and supporting artifacts. While the project repository will be hosted on GitHub, it will not contain the full

testbed environment itself, as the operational testbed spans multiple EC2 instances and cannot be replicated directly within source control. Instead, GitHub will serve as the centralized location for all documentation, architecture references, configuration templates, and copies of every script used to deploy or operate the testbed.

9.0 Limitations and Known Issues

Although the system is stable and all core components function as intended, the realism of the synthetic log data remains a limitation. The generated events are structurally correct and ECS-compliant, but they lack deeper behavioral patterns found in real SOC environments. Hostnames, usernames, and source IPs are randomly combined rather than reflecting consistent user–host relationships, role-based access patterns, or long-term behavioral baselines. Event timing is uniformly spaced, adversarial activity does not form multi-stage attack chains, and benign noise lacks the irregularity and variance seen in production telemetry. As a result, the dataset is useful for controlled testing but does not fully represent the complexity, unpredictability, and correlation-rich nature of enterprise-scale logs.

The Conversational AI interface also exhibits notable limitations, primarily in its ability to generate accurate and executable Elasticsearch JSON DSL queries. Errors commonly arise from incorrect field names, misplaced ECS hierarchy, misunderstandings of dataset structure, and incomplete boolean logic. These issues stem largely from insufficient prompt engineering and the model's lack of contextual awareness of Elasticsearch schemas. Additionally, because the CAI does not receive execution feedback from Elasticsearch, it cannot refine or self-correct invalid queries. These constraints limit the CAI's current effectiveness, but they can be fixed through improved prompt design, schema guidance, and iterative refinement.

10.0 Backup and Recovery Plan

The backup strategy involves automated daily exports of the GitHub repository to an encrypted AWS S3 bucket using AWS Backup services, with a 30-day retention policy, supplemented by periodic EBS snapshots and AMI creations for EC2 instance configurations to allow quick restores and recreation. Manual backups will be made before any architecture changes are made.

11.0 Change Log

Ver.	Date	Description of Change	Author
v1.0	2025-10-07	Initial setup of documentation	Prof. Littleton
v1.1	2025-10-22	Planned the testbed.	Koi McFarland
v1.2	2025-10-31	Remade Diagram / Improved Design	Koi McFarland
v1.3	2025-11-12	Remade Diagram / Implementation	Koi McFarland
v1.4	2025-12-17	Removed “Validation and Testing” section	Koi McFarland

12.0 Mentor Review / Approval

Reviewer	Role	Date	Notes
Faculty Advisor			
DoD/ACE Mentor			